AI-TOURISM HACKATHON 2026 · TBI-GEU

# HOI

## Hospitality Operations Intelligence

Technology stack, architecture, features and differentiation

**Prepared for:** the team, mentors and judges

**Date:** 13 October 2026

**Status:** working prototype on synthetic data (TRL 4), ready for a real-property pilot

> All data in this project is **synthetic**. Every accuracy figure comes from a seeded synthetic hotel, not a real property. Risk flags exist to **offer support**, never for adverse action.

# Contents

1. Executive summary ..... 4
1.1 What HOI is ..... 4
1.2 Key pointers ..... 4
1.3 The one-line pitch ..... 4
2. The problem and who it is for ..... 5
2.1 The problem ..... 5
2.2 Who uses HOI ..... 5
3. What we are making: product surfaces ..... 6
4. Feature catalogue ..... 7
4.1 Forecast and overview ..... 7
4.2 Retention (attrition risk) ..... 7
4.3 Roster ..... 7
4.4 Live dispatch and proof of work ..... 8
4.5 Rooms and guests (inspired by eZee's guest self-service and fast checkout) ..... 8
4.6 Inventory and purchasing ..... 8
4.7 SaaS platform ..... 9
5. What makes HOI different ..... 10
5.1 Positioning against competitor categories ..... 10
5.2 Where we are weaker (say it before a judge does) ..... 11
6. Architecture ..... 12
6.1 End-to-end flow ..... 12
7. Technology stack in detail ..... 13
7.1 Languages and runtime ..... 13
7.2 ML layer (team ML repo) ..... 13
7.3 ML / ops API (team backend, Render) ..... 13
7.4 HOI platform API (ours, backend/) ..... 14
7.5 Manager dashboard (ours, frontend/) ..... 14
7.6 Staff app and guest page (ours, served by the platform API) ..... 15
7.7 Marketing website (ours, website/) ..... 15
7.8 Quality, testing and DevOps ..... 15
8. Security and privacy ..... 17
9. Models and results (synthetic data) ..... 18
10. Testing and verification ..... 19
11. Deployment and operations ..... 20
11.1 Ways to run ..... 20

11.2 Key settings ..... 20
12. Design choices and trade-offs ..... 21
13. Roadmap ..... 22
13.1 Before real hotel data ..... 22
13.2 Product ..... 22
14. Glossary ..... 23
Appendix A. Repository structure ..... 24
Appendix B. Platform API endpoints ..... 25

# 1. Executive summary

## 1.1 What HOI is

HOI is an **operations and retention layer for small hill hotels and homestays**, starting in Uttarakhand. It sits next to the hotel's property-management system (PMS, for example eZee) and runs the day-to-day work of the staff: it forecasts demand, builds fair rosters, spots staff who may need support and explains why, turns that into actions and a better roster, sends live requests to the right worker with photo proof, and runs room turnaround and stores.

It is built as a **multi-tenant SaaS product**: HOI staff onboard hotel groups from a platform console, each hotel group's data is isolated, and managers, frontline employees and guests each get their own app.

## 1.2 Key pointers

- **Four user-facing surfaces:** manager dashboard (Streamlit), staff phone app (installable web app, Hindi/English, live camera), guest QR request page, marketing website.
- **Two back ends:** the team's ML/operations API (forecast, attrition, OR-Tools roster solver, tasks, closed loop, voice) and our HOI platform API (accounts, hotel groups, dispatch, rooms, photos, inventory with GST, audit).
- **Four models:** occupancy forecast, attrition risk (with a contract-date rule for seasonal staff), workload, pulse. Each is reported beside a simple baseline.
- **One constraint solver:** Google OR-Tools CP-SAT for weekly rosters, with a greedy fallback, which refuses to publish a roster that breaks a rule.
- **Six roles:** platform admin, owner/GM, HR, department head, storekeeper, employee. Each sees only its pages; the API re-checks permissions on every call.
- **Security:** PBKDF2 password hashing, 30-minute access tokens with rotating refresh tokens and reuse detection, rate limits, Content-Security-Policy, tenant isolation, audit log, production configuration guard.
- **Privacy by design:** staff are identified by ID only; imports with name, gender, religion, caste, health or family columns are rejected; guests give no personal data; employees never see risk scores.
- **Quality:** 28 API tests, a 22-page smoke test run as real signed-in roles, browser end-to-end tests (Playwright with Chrome), ruff lint and format, a CI pipeline.

## 1.3 The one-line pitch

> HOI is the only system we found that closes the loop from individual attrition risk to an actual roster change and back, and now also dispatches the day's work to the right person with proof. It is an operations and retention layer, not a PMS or payroll system, and every number is from synthetic data until a real pilot proves otherwise.

# 2. The problem and who it is for

## 2.1 The problem

- Small hill hotels plan staff by hand on paper or WhatsApp, and discover resignations late, usually during peak season.
- Demand swings sharply: Char Dham pilgrimage peaks in May–June, festivals (Dussehra, Diwali, Christmas–New Year) spike weekends, and February is a trough.
- Overtime, missed weekly offs and pay gaps build up quietly until good staff leave, and replacing them costs weeks of training.
- Guest requests (towels, AC repair, tea) travel by phone calls and shouting across floors; nobody can prove a job was done or how long it took.
- Stores are run on par levels that ignore the coming week's occupancy, so busy weeks run short and slow weeks over-stock.

## 2.2 Who uses HOI

| User | Their job | What HOI gives them |
|---|---|---|
| Owner / general manager | Profit, service quality, keeping staff | Overview of cost, coverage and risk; ROI view; every module |
| HR | Retention, fairness | Risk list with reasons, what-if simulator, closed loop, intervention tasks |
| Department head (housekeeping, F&B, maintenance) | Run the shift | Roster, live dispatch board, rooms board, supply requests |
| Storekeeper | Stock and purchasing | Requests, purchase orders with GST, receipts, counts, wastage |
| Frontline employee | Do the work, get rest | Phone app: own shifts, job pings, photo proof, supplies, Hindi/English |
| Guest | Get help fast | Scan the room QR, tap a request; no app, no login, no personal data |
| HOI platform admin | Onboard and support hotels | Console: create hotel groups, plans, modules, suspend, audit |

# 3. What we are making: product surfaces

| Surface | Users | What it does | Technology |
|---|---|---|---|
| Manager dashboard | Owner, HR, department heads, storekeeper, platform admin | 15 hotel pages: overview, risk, employee, what-if, roster, closed loop, dispatch, rooms, worker preview, inventory, tasks, voice, setup, admin; plus a 2-page platform console | Streamlit, Plotly, pandas |
| Staff app (/worker/) | Frontline employees | Today's shift and week, live job pings with countdown and vibration, accept/decline, live-camera before/after photos, supply requests, Hindi/English, installable | HTML/CSS/JavaScript PWA served by the platform API |
| Guest QR page (/guest/) | Hotel guests | One random link per room; bilingual menu of requests that become auto-dispatched jobs; live status | HTML/CSS/JavaScript served by the platform API |
| Marketing website | Prospects, judges | Product story, hill-hotel focus, comparison by category, plans, FAQ, links to the demo | Static HTML/CSS/JavaScript |
| Platform API | All apps (machine users) | Accounts, tenants, dispatch engine, rooms, photos, inventory, GST, audit | FastAPI, SQLite, PyJWT, Pillow |
| ML / ops API | Dashboard | Forecast, attrition score and rescore, roster solver, tasks and reminders, closed loop, metrics, voice | FastAPI, LightGBM, scikit-learn, OR-Tools, APScheduler |

# 4. Feature catalogue

Features by module. Everything listed here is built and running unless marked as roadmap.

## 4.1 Forecast and overview

| Feature | What it does | Detail / honesty note |
|---|---|---|
| Occupancy forecast | Next 1, 7 and 30 days with an uncertainty band and high/low demand alerts | The 80 % band covered only ~68 % of days in backtests; the UI says so |
| Overview dashboard | Overtime cost, understaffed shifts, task on-time rate, flagged staff, value at stake | ROI uses an editable replacement-cost assumption, labelled as an assumption |
| Department load | Required vs scheduled hours per department, colour-coded | From the solved roster |

## 4.2 Retention (attrition risk)

| Feature | What it does | Detail / honesty note |
|---|---|---|
| Risk list | Every employee ranked into high / watch / low bands with the top reason | Bands are rankings, not predictions; permanent staff by model rank, seasonal staff by contract date |
| Driver explanations | Per-person log-odds contributions grouped into 7 driver categories | Each driver labelled supported / weakly supported / unsupported |
| Employee detail | 12-week trajectory, plain-language reasons, matched playbook actions → tasks | Demo heroes only have trajectories (the API serves the current week) |
| What-if simulator | Re-score a person with protected weekends or changed features | A simulation, not a causal claim; small effects shown as small |
| Closed loop | Score → create interventions → re-solve roster with protected weekends → re-score | Reports the measured change honestly (largest 0.0123 in the demo week) |

## 4.3 Roster

| Feature | What it does | Detail / honesty note |
|---|---|---|
| Constraint roster | Weekly shifts from the forecast under hard rules (leave, rest, weekly off, skills, headcount) | OR-Tools CP-SAT, 10 s limit, greedy fallback |
| Honest infeasibility | When a week cannot be staffed, lists the exact short department/day/shift | Never publishes a partial roster |
| Fairness and trade-offs | Fairness score distribution, soft goals not met, baseline vs protected diff | |
| Roster to staff | Managers' roster is published so each employee sees only their own shifts | Platform API /me/shifts |

## 4.4 Live dispatch and proof of work

| Feature | What it does | Detail / honesty note |
|---|---|---|
| Auto-assign | Each request is offered to the best on-shift worker of the right department | Score = 100 − 25·open jobs − 8·done this shift − 6·floor distance − retention guard |
| Retention guard | Workers flagged high/watch for overload get fewer extra jobs (−15 / −6) | Ties dispatch to the retention model |
| Timed offers | Ping with countdown; decline or timeout moves to the next worker; nobody left → supervisor | Expiry enforced by the server |
| Photo proof | Live-camera before photo to start, after photo to finish | Server time + job + worker stamped on the image; SHA-256 tamper check; cannot detect a photo of a screen |
| Voice to task | A voice note becomes a structured, department-routed job | The live backend currently uses a cached reference transcript |
| Task board | Kanban with 50 % / 90 % reminders, escalation and a demo clock | From the ML/ops API |

## 4.5 Rooms and guests (inspired by eZee's guest self-service and fast checkout)

| Feature | What it does | Detail / honesty note |
|---|---|---|
| Room status board | Ready / occupied / dirty / cleaning / inspection / out of order for every room | 45 demo rooms on 4 floors |
| Checkout → cleaning job | Checkout auto-creates a cleaning job; starting and finishing it moves the room status | Room waits for supervisor inspection before it is sellable |
| Inspection queue | One-click Pass or Re-clean (re-clean creates a new job) | |
| Out of order | Marks a room unsellable and raises a maintenance job | Back in service triggers a clean |
| Guest QR requests | Guest scans the room code, taps towels / AC / tea / …, sees status | No login, no personal data; per-room and per-IP limits |

## 4.6 Inventory and purchasing

| Feature | What it does | Detail / honesty note |
|---|---|---|
| Catalogue and locations | Items with unit, cost, par, reorder point, supplier, lead time, usage per occupied room | Main store, floor pantry, kitchen, maintenance store |
| One ledger | Every receipt, issue, transfer, wastage and count adjustment is a ledger row | Invariant: stock always equals the sum of movements; negative stock refused |
| Requisitions | Staff or departments request; storekeeper issues fully or partly, or rejects | Employees request from their phone |
| Purchase orders | Draft → approved → sent → partial → received (GRN) | Partial receipts supported |

| Feature | What it does | Detail / honesty note |
|---|---|---|
| Forecast-aware reorder | Order quantity from usage per occupied room × forecast occupancy over 7 days + lead time | Flags items that look fine on static par but will run short |
| GST | HSN and GST rate per item; supplier GSTIN check; CGST+SGST or IGST split; printable PO | Rates are placeholders to confirm with an accountant; the PO is not a tax invoice |
| Counts and analytics | Cycle counts with variance value; consumption by department; cost per occupied room; wastage; linen par | |

## 4.7 SaaS platform

| Feature | What it does | Detail / honesty note |
|---|---|---|
| Platform console | Create hotel groups with plan, modules, first property and a one-time owner invite; suspend/reactivate | Every action audited |
| Onboarding wizard | Owner: organisation → properties → staff import → data source test → team → done | Privacy guard on import |
| Plans and modules | Starter / Growth / Chain; modules switch whole page groups on or off | No prices shown (set with pilot partners) |
| Team and access | Invite codes per role, enable/disable users | No email addresses stored |
| First-time experience | One-click demo roles, welcome dialog, 9-step guided tour | Demo logins disabled in production |
| Audit log | Sign-ins, invites, plan changes, workspace access, dispatch and inventory actions | Exportable |

# 5. What makes HOI different

| # | Differentiator | Why it matters | Evidence in the build |
|---|---|---|---|
| 1 | Closed loop: risk → interventions → re-solved roster → re-score | Schedulers (Legion, NymbleUp) do not score individual attrition; attrition tools (inFeedo, Darwinbox) do not change rosters | Closed-loop page and API; honest small-effect reporting |
| 2 | Retention-aware dispatch | Avoids piling extra jobs on people already flagged for overload | Dispatch ranking with a 'Why this worker?' breakdown |
| 3 | Forecast-aware purchasing | Stock follows the same forecast as the roster | 4 demo items caught that static par misses |
| 4 | Honest, explained outputs | Each reason labelled by support level; models beside baselines; calibrated language | Risk list, model cards, UI copy |
| 5 | A roster solver that will not lie | Infeasible weeks are explained, never silently broken | Roster infeasible banner |
| 6 | India hill-hotel native | Pilgrimage season, festivals, seasonal contracts, GST, Hindi | Data generator, inventory GST, staff app |
| 7 | Frontline-first staff app | Live-camera proof, vibration pings, works on cheap Android phones | Verified end-to-end in Chrome |
| 8 | Privacy enforced in code | Sensitive columns rejected server-side; workers never see risk bands | API tests |

## 5.1 Positioning against competitor categories

| Capability | HOI | Scheduling / WFM | HR attrition analytics | Hotel task apps | Hotel PMS (e.g. eZee) |
|---|---|---|---|---|---|
| Demand forecast → roster | Yes | Yes | No | No | No |
| Individual, explained attrition risk | Yes | No | Yes | No | No |
| Risk → action → roster → re-score loop | Yes | No | Partial | No | No |
| Live dispatch with photo proof | Yes | Partial | No | Yes | Partial |
| Guest service requests | Yes (QR) | No | No | Partial | Yes |
| Forecast-aware purchasing | Yes | No | No | No | No |
| Reservations, rates, channel manager, POS | No (integrate) | No | No | No | Yes |
| Payroll / attendance system of record | No | Partial | Yes | No | Partial |
| Proven on real hotel data | Not yet | Yes | Yes | Yes | Yes |

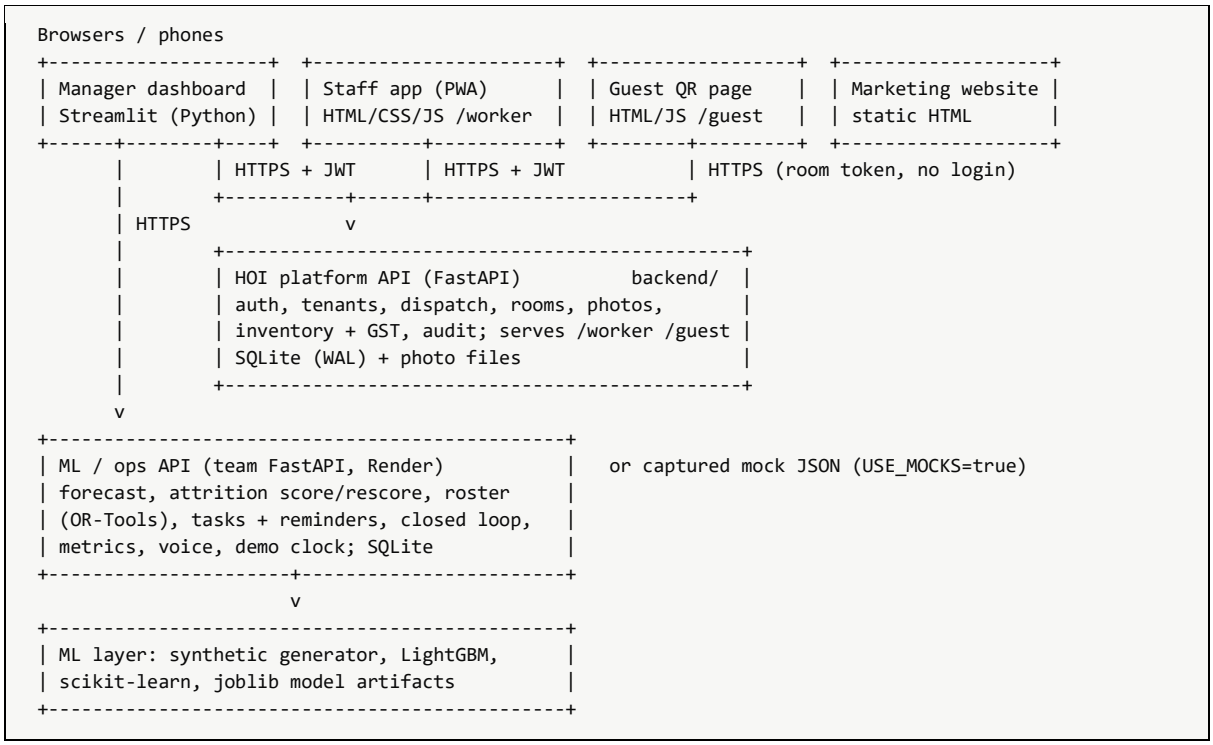
Based on public vendor material and review sites, October 2026. 'Partial' means limited or not publicly evidenced.

## 5.2 Where we are weaker (say it before a judge does)

- No real-hotel validation yet: every number is synthetic.
- Not a PMS, payroll or attendance system of record; HOI integrates with those.
- Tested at one-property scale; the roster solver hits its 10-second limit in ~40 % of stress-test scenarios.
- The pulse (engagement) score is simulated, not measured from real check-ins.
- No SSO, billing, or security certification (ISO 27001 / SOC 2 / DPDP review) yet.

# 6. Architecture

A modular system with clear boundaries: two APIs and four front ends. The manager dashboard is the only client of the ML/ops API; the platform API never trusts a client-supplied hotel id (it comes from the signed token).




## 6.1 End-to-end flow

1. **Data:** a seeded generator creates occupancy, weather, staff, weekly behaviour and exits (same seed → identical files).
2. **Forecast:** the model predicts occupancy with a band and alerts.
3. **Workload:** forecast converts to required hours per department.
4. **Roster:** the solver assigns shifts under hard rules and soft goals; the dashboard publishes it to staff and to the dispatch engine.
5. **Work:** guest QR requests, voice notes and managers create jobs; the best on-shift worker is pinged and proves the work with photos.
6. **Risk:** weekly scoring ranks every employee with reasons.
7. **Fix:** each flagged person gets matched playbook actions as manager tasks; the roster re-solves with protected weekends.
8. **Measure:** re-score and show what changed, including when the change is small.

# 7. Technology stack in detail

Versions are those installed and tested on the build machine, except rows marked 'team ML repo' (code not in this workspace; taken from the project documents).

## 7.1 Languages and runtime

| Component | Technology / design | Version / note |
|---|---|---|
| Main language | Python, one language for data, models, APIs and dashboard | 3.12.4 (ML repo plan: 3.13) |
| Browser apps | HTML5, CSS3, ES2020 JavaScript, no framework, no build step | Strict CSP possible |
| Environments | venv per app, pip requirements files | |

## 7.2 ML layer (team ML repo)

| Component | Technology / design | Version / note |
|---|---|---|
| Synthetic data | numpy seeded Generator (rng.spawn), pandas, PyYAML | 45 rooms, ~55 staff, 5 demo heroes kept out of training |
| Occupancy forecast | LightGBM + seasonal-naive ensemble, split-conformal interval | MAPE 6.7 / 7.2 / 10.3 % vs naive ~13.7 % |
| Attrition (permanent) | scikit-learn logistic regression, Platt calibration; LightGBM compared | ROC-AUC 0.691 vs rules 0.542; LightGBM 0.652 |
| Attrition (seasonal) | Contract-date rule | ROC-AUC 0.736 |
| Explanations | Logistic contributions in log-odds, grouped into 7 driver categories with support levels | Same model for score and explanation |
| Workload, pulse | LightGBM / HistGradientBoosting regressors | Workload MAE 0.053 vs 0.073; pulse 0.250 vs 0.257 |
| Packaging | joblib + metrics.json, model_card.md, feature_list.json, dataset SHA-256 | Shown on the Admin page |
| Evaluation | Time-based splits with a 90-day gap, oracle ceiling, multi-seed checks | 58 validation checks, 330+ tests |
| Challenger assessed | Google TimesFM 2.5 (Apache-2.0) | 5.8 % vs 11.0 % MAPE on our synthetic series; ~1.1 GB RAM → batch job; 3.0 is non-commercial |

## 7.3 ML / ops API (team backend, Render)

| Component | Technology / design | Version / note |
|---|---|---|
| Framework | FastAPI + Pydantic, uvicorn | |
| Roster solver | Google OR-Tools CP-SAT; hard and soft constraints; 10 s limit; greedy fallback | |
| Background jobs | APScheduler: reminders at 50 % / 90 %, escalation, demo clock | |

| Component | Technology / design | Version / note |
|---|---|---|
| Storage | SQLite via SQLAlchemy | Free hosting tier sleeps; restart resets data |
| Voice | Swappable providers (Groq Whisper, Sarvam, faster-whisper) + glossary-based extraction | Live demo uses a cached reference transcript |
| Mock mode | MOCK=1 fixtures; the dashboard ships fixtures captured from the live API | |

## 7.4 HOI platform API (ours, backend/)

| Component | Technology / design | Version / note |
|---|---|---|
| Web framework | FastAPI on Starlette, Pydantic v2 request validation | fastapi 0.143.0 · starlette 1.7.0 · pydantic 2.14.1 |
| Server | uvicorn ASGI, one worker | 0.54.0 |
| Storage | SQLite in WAL mode, transactions, additive migrations; photos on disk | Python stdlib sqlite3 |
| Code layers | db → services (pure logic, typed errors) → routers (HTTP) → main | |
| Passwords | PBKDF2-HMAC-SHA256, 200 000 iterations, per-user salt, constant-time compare, lockout after 5 failures | Python stdlib |
| Tokens | JWT HS256 access (30 min) + opaque refresh (14 days, hashed, rotated, reuse revokes the family) | PyJWT 2.15.1 |
| Authorization | 6 roles, per-route guards, tenant from the token only, audited impersonation | |
| Rate limiting | Sliding window per IP and per user on login, sign-up, refresh, demo and guest endpoints | In-process |
| Security headers | CSP on web apps, nosniff, no-referrer, frame-deny, HSTS in production, request IDs | |
| Images | Pillow: orientation fix, resize, server-time stamp, JPEG, SHA-256 | Pillow 12.3.0 |
| QR codes | segno SVG per room | 1.6.6 |
| Uploads | python-multipart | 0.0.32 |
| Config guard | HOI_ENV=production refuses weak secrets, demo logins, wildcard CORS, missing admin password | |
| Ops endpoints | /health, /ready, OpenAPI docs (hidden in production) | |

## 7.5 Manager dashboard (ours, frontend/)

| Component | Technology / design | Version / note |
|---|---|---|
| UI framework | Streamlit: st.navigation router, fragments for live refresh, dialogs, forms, data editor | 1.65.0 |
| Charts | Plotly: forecast bands, driver bars, trajectories, histograms | 7.1.0 |
| Data | pandas | 3.0.6 |

| Component | Technology / design | Version / note |
|---|---|---|
| HTTP | requests (ML API, remote platform API); FastAPI TestClient/httpx for the embedded platform API | requests 2.34.2 · httpx 0.28.1 |
| Config | python-dotenv: USE_MOCKS, API_BASE_URL, PLATFORM_API_URL (embedded or URL) | 1.2.4 |
| Structure | app.py router · login · console/ · pages/ (15) · employee/ (4) · lib/ (clients, models, auth, access, theme, charts, tour) | |
| Session | Tokens in session state; refresh token in a SameSite=Strict cookie (survives browser refresh); silent refresh on 401 | |
| Design system | Custom CSS: Inter Tight, pill navigation, outline status cards, motion with reduced-motion support, WCAG-checked colours | From the Train.png references |

## 7.6 Staff app and guest page (ours, served by the platform API)

| Component | Technology / design | Version / note |
|---|---|---|
| Staff app | Vanilla JS progressive web app: manifest, service worker (offline shell), installable, bottom tab bar, 2-second polling for pings, vibration | |
| Camera | getUserMedia rear camera → canvas → JPEG upload; no file picker | Live camera only |
| Language | Hindi / English string tables, remembered per phone | Hindi to be reviewed by native speakers |
| Session | Access token in memory; refresh token in localStorage ('keep me signed in') or sessionStorage | Strict CSP limits XSS risk |
| Guest page | Vanilla JS; room token from the QR URL; bilingual menu; status refresh every 15 s | |
| Shared look | shared/base.css design tokens; system fonts (keeps CSP to 'self') | |

## 7.7 Marketing website (ours, website/)

| Component | Technology / design | Version / note |
|---|---|---|
| Page | Single static index.html: HTML/CSS/vanilla JS | |
| Typography | Inter Tight from Google Fonts | |
| Motion | IntersectionObserver reveal animations, respects reduced motion | |
| Responsive | Works from 390 px phones to desktop | |

## 7.8 Quality, testing and DevOps

| Component | Technology / design | Version / note |
|---|---|---|
| API tests | pytest + FastAPI TestClient: 28 tests | pytest 9.1.1 |

| Component | Technology / design | Version / note |
|---|---|---|
| Dashboard tests | Streamlit AppTest: 22 pages rendered as real signed-in roles + role-filter checks | |
| End-to-end | Playwright driving installed Chrome (phone viewport, fake camera) | 1.63.0 |
| Lint / format | ruff: pycodestyle, pyflakes, isort, bugbear, pyupgrade; ruff format | 0.17.0 |
| CI | GitHub Actions: install → lint → format check → API tests → dashboard smoke test | |
| Containers | Dockerfiles (python:3.12-slim, non-root, health checks) + docker-compose | Not build-tested on this machine (no Docker) |

# 8. Security and privacy

| Area | Control | Status |
|---|---|---|
| Passwords | PBKDF2-SHA256 (200k iterations, salted), constant-time compare, equal timing for unknown users, lockout after 5 failures | Built, tested |
| Sessions | 30-minute access tokens; 14-day refresh tokens stored only as hashes, rotated on use; reuse of an old token revokes all sessions; sign-out revokes | Built, tested |
| Sign-up | Only with a single-use invite code that fixes hotel and role; employees must give their employee ID | Built, tested |
| Authorization | Role guard on every route; hotel id only from the signed token; cross-hotel ids rejected | Built, tested |
| Platform admin | Reaches hotel data only via audited impersonation | Built, tested |
| Abuse limits | Rate limits on login, sign-up, refresh, demo and guest endpoints; per-room guest request limit | Built, tested |
| Browser security | CSP without inline scripts on staff and guest apps; nosniff; no-referrer; frame-deny; HSTS in production | Built, tested |
| Production guard | Refuses to start with a weak secret, demo logins, wildcard CORS or no admin password | Built, tested |
| Staff data | Employee ID only; imports with sensitive columns (name, gender, religion, caste, health, family, phone, email…) rejected server-side | Built, tested |
| Guest data | No login and no personal data; random per-room token | Built |
| Purpose limitation | Risk flags are for support, not punishment; employees never see their own or others' risk bands | Built (UI and API) |
| Photo proof | Server timestamp, job and worker stamped; SHA-256 re-verified on read | Built; cannot detect a photo of a screen |
| Audit | Sign-ins, invites, plan/module changes, workspace access, dispatch and inventory actions | Built |
| Still to do | SSO/OIDC, encryption at rest, backups, DPDP Act review, security certification | Roadmap |

# 9. Models and results (synthetic data)

| Model | Result | Baseline | Honest note |
|---|---|---|---|
| Occupancy forecast (MAPE, 1/7/30 days) | 6.7 % / 7.2 % / 10.3 % | Seasonal-naive ~13.7 % | 80 % interval covered only ~70 / 68 / 56 % of days |
| Attrition: permanent resignations | ROC-AUC 0.691, PR-AUC 0.119 | Rules 0.542 / 0.071; LightGBM 0.652 | Only ~31 leavers in test years; modest and noisy |
| Attrition: seasonal contract ends | Contract-date rule ROC-AUC 0.736, PR-AUC 0.316 | Rules 0.423 | Dominated by the contract date |
| Workload (MAE) | 0.053 | Naive 0.073 | Clear gain |
| Pulse (MAE) | 0.2498 | Naive 0.2567 | Marginal gain |
| TimesFM 2.5 challenger (our test) | 5.8 % MAPE (30 days) | Seasonal-naive 11.0 % | Our own synthetic series, not project data; ~1.1 GB RAM |

Evaluation discipline: time-based splits with a 90-day gap (never random), an oracle ceiling (permanent-staff resignation tops out at ROC-AUC 0.906 even with hidden features), multi-seed checks, and probability calibration on a validation period.

# 10. Testing and verification

| Check | How | Result |
|---|---|---|
| API behaviour | 28 pytest tests: auth, lockout, refresh rotation and reuse, RBAC, tenant isolation, suspension, impersonation, privacy guard, dispatch lifecycle, photos, rooms, guest limits, GST, inventory ledger, production config | All pass |
| Dashboard pages | 22 pages rendered as real signed-in roles through the API + 3 role-filter checks | All pass |
| Live ML backend | Closed loop (32 s, 12 tasks), task ack, what-if, voice demo clip, overview | Pass |
| Staff app end-to-end | Chrome at phone size with fake camera: ping → accept → before photo → after photo → done; supplies; Hindi; reload keeps session | Pass, zero JS/CSP errors |
| Guest page | QR link → request → job dispatched → status | Pass |
| Dashboard session | Sign in → Rooms → checkout → browser refresh → still signed in | Pass |
| Production config | Safe settings boot (admin login works, no demo, docs hidden); unsafe settings refused | Pass |
| Accessibility | WCAG contrast measured and fixed; visible keyboard focus | Fixed; no screen-reader audit yet |
| Not verified | Docker builds; real phones and networks; real hotel data; multi-instance deployment | Open |

# 11. Deployment and operations

## 11.1 Ways to run

| Mode | Command | Use for |
|---|---|---|
| Demo (one process) | cd frontend; streamlit run app.py | Laptop demo; platform API runs inside the dashboard |
| Full stack | cd backend; uvicorn hoi_platform.main:app --port 8100<br>cd frontend; PLATFORM_API_URL=http://localhost:8100 streamlit run app.py | Staff app and guest QR pages (need the API as a service) |
| Containers | cp .env.example .env; docker compose up --build | Servers (files written, not build-tested here) |

## 11.2 Key settings

| Variable | Meaning | Production value |
|---|---|---|
| HOI_ENV | development or production (production enforces safe settings) | production |
| HOI_JWT_SECRET | Signing secret for access tokens | 48+ random characters |
| HOI_ADMIN_PASSWORD | Creates the platform admin on first start | 12+ characters |
| HOI_DEMO_LOGIN / HOI_SEED_DEMO | One-click demo accounts / demo hotel | false / false |
| HOI_CORS_ORIGINS | Browser origins allowed to call the API | Explicit list |
| HOI_PUBLIC_BASE_URL | Public URL printed into room QR codes | https://… |
| USE_MOCKS / API_BASE_URL | Dashboard: captured demo data or the live ML API | false / ML API URL |
| PLATFORM_API_URL / PLATFORM_PUBLIC_URL | Where the dashboard and phones reach the platform API | Service URL / public URL |

# 12. Design choices and trade-offs

| Choice | Benefit | Cost / when to change |
|---|---|---|
| Streamlit for managers | Python-only, fast to build, charts built in | Weak on phones and fine-grained UI; React is the upgrade path |
| Plain JavaScript for staff and guests | Fast on cheap phones, no build, strict CSP | More hand-written code; fine at this size |
| SQLite | Zero setup, one file, instant demo reset | One server only; move to Postgres (plain SQL schema) |
| In-process rate limiter | No extra service | Needs Redis with more than one API instance |
| Embedded platform API | One command runs the demo | Staff and guest apps need the API as a separate service |
| Logistic regression for attrition | Better on our data, calibrated, explainable | Our synthetic hazard is logistic; re-check on real data |
| Custom JWT auth | Full control, no vendor lock-in | Production should add SSO/OIDC |
| Integrate with a PMS instead of building one | Focus on staff operations and retention | Needs PMS APIs (room status, occupancy actuals) |

# 13. Roadmap

## 13.1 Before real hotel data

- Real-property pilot with the hospitality-management student's confirmed pay, workload and rest-hour values.
- Postgres, object storage for photos, backups, SSO/OIDC, TLS, monitoring; DPDP Act review.
- Build and test the Docker images; test the staff app on real low-end Android phones and weak networks.

## 13.2 Product

- PMS integration (e.g. eZee) for room status and occupancy actuals.
- Telegram / WhatsApp push for job pings; native wrappers if needed.
- Multi-week intervention memory so the closed loop shows trends, not one week.
- Booking-pace feature for short-horizon forecasts; TimesFM 2.5 as a batch challenger after a fair backtest.
- Multi-property roll-up dashboards; payroll/attendance integration; more languages (Garhwali, Nepali).

# 14. Glossary

| Term | Meaning |
|---|---|
| PMS | Property-management system: reservations, rates, check-in/out, guest invoices (e.g. eZee Absolute) |
| Band (high / watch / low) | A within-week ranking of who to check in with; not a prediction that someone will leave |
| Driver | A reason contributing to a person's risk score (e.g. overtime overload), with its support level |
| Closed loop | Score → act → re-roster → re-score, measuring the effect |
| CP-SAT | Google OR-Tools constraint solver used for rosters |
| Dispatch | Offering a job to the best on-shift worker with a timed ping |
| Par level / reorder point | Target stock level / level at which to reorder |
| GRN | Goods received note: recording a delivery against a purchase order |
| HSN / GSTIN | Harmonised commodity code for GST / a business's 15-character GST number |
| PWA | Progressive web app: a website that installs and works like a phone app |
| JWT / refresh token | Short-lived signed access token / long-lived token used to get new access tokens |
| CSP | Content-Security-Policy: browser rule that blocks injected scripts |
| MAPE / ROC-AUC | Forecast error in percent / how well a score ranks leavers above stayers (0.5 = chance) |
| TRL 4 | Technology readiness level 4: validated in a lab (synthetic) environment |

# Appendix A. Repository structure

```
INNOV8/
  backend/            HOI platform API
    hoi_platform/     main, config, db, security, deps, schemas, ratelimit, seed
      services/       tenants, users, sessions, dispatch, rooms, photos, inventory, errors
      routers/        auth, platform, workspace, dispatch, rooms, public, me, inventory
      static/worker   staff app (PWA)      static/guest  guest QR page
    tests/            28 pytest tests      Dockerfile, requirements.txt
  frontend/           Streamlit dashboard
    app.py, login.py, home.py  console/ (2)  pages/ (15)  employee/ (4)
    lib/              api_client, platform, auth, access, tenancy, data, models, theme, charts, ...
    mocks/            fixtures captured from the live ML API
    tests/smoke.py    22-page role smoke test      Dockerfile, requirements.txt
  website/index.html  marketing site
  docs/               tech stack, scope, competitors, TimesFM, UX review, contract, plan
  docker-compose.yml  .env.example  pyproject.toml (ruff)  .github/workflows/ci.yml
```

# Appendix B. Platform API endpoints

| Area | Endpoints |
|---|---|
| Auth and sessions | POST /auth/login · /auth/register · /auth/refresh · /auth/logout · GET /auth/demo-accounts · POST /auth/demo/{user} · GET /auth/me · PATCH /auth/me/prefs · POST /auth/impersonate/{tid} |
| Platform console | GET/POST /platform/tenants · PATCH/DELETE /platform/tenants/{id} · POST /platform/tenants/{id}/invites · GET /platform/audit |
| Workspace | GET/PATCH /workspace · GET/POST /workspace/properties · GET /workspace/staff · POST /workspace/staff/import · GET /workspace/users · PATCH /workspace/users/{id} · GET/POST /workspace/invites · GET /workspace/audit |
| Dispatch | PUT/GET /dispatch/context · PUT /dispatch/roster · POST/GET /dispatch/jobs · POST /dispatch/jobs/{id}/accept\|decline\|start\|complete\|offer\|retry\|link-task · POST/GET /dispatch/jobs/{id}/photos · GET /dispatch/photos/{id} · POST /dispatch/reset |
| Rooms | GET/POST /rooms · POST /rooms/bulk · POST /rooms/{id}/checkout\|check-in\|inspect\|out-of-order\|back-in-service · GET /rooms/{id}/qr.svg |
| Guest (public) | GET /public/guest/{token} · POST /public/guest/{token}/requests |
| Staff | GET /me/shifts |
| Inventory | GET/POST /inventory/items\|locations\|suppliers · GET /inventory/stock\|moves · POST /inventory/receive\|issue\|waste\|transfer · GET/POST /inventory/requests · POST /inventory/requests/{id}/fulfil\|reject · GET/POST /inventory/pos · POST /inventory/pos/{id}/status\|receive · GET /inventory/pos/{id}/document · GET/POST /inventory/counts · POST /inventory/counts/{id}/post · GET /inventory/suggestions · GET /inventory/analytics |
| Operations | GET /health · GET /ready · /docs (development only) |